

KAFRIADA CORE

The architecture derived from what the system is — a national identity authority that holds money — rather than from what is convenient to build. Presentation, domain and work are separate tiers. The money core is isolated by the database, not by convention.

SUPERSEDES	API TIER	PRESENTATION	DATA
Draft v1 (single-tier)	FastAPI · Python 3.12	Next.js 15 as BFF	PostgreSQL 16 · primary + replica

STATUS
Summary for CTO approval

01

Position

What changed between Draft v1 and this document, and why.

Draft v1 derived its architecture from delivery capacity — three developers, eight weeks, one deploy unit. That is the wrong derivation order and I am correcting it. **The system's properties determine the architecture. Delivery capacity is a risk to be managed against that architecture, not an input to it.**

Restated from the business, which is where it should have started: KAFRIADA CORE is not an app that happens to register footballers. It is **an identity authority for African sport that also holds other people's money**. An identity authority issues permanent, non-reusable identifiers that third parties are expected to trust and, eventually, to query programmatically. A system holding other people's money needs a money core that is isolated, auditable, and provably unable to be written by anything outside itself.

Those two sentences have architectural consequences, and they are the ones that produce this design:

- An identity authority's **API is the product**, not a private implementation detail of a web UI. Federations, state sports commissions, a future native app and the scouting tier in the GDOC are all API consumers. The contract must be first-class, versioned and machine-checked from day one — because you cannot retrofit a contract onto callers who already depend on your internals.
- A money core belongs in **its own consistency and privilege domain**. "It is a folder in our application" is not a position you can defend to an auditor, a bank, or the Central Bank. "It is a schema no other database role can write to" is.
- The public QR profile, the transactional API and the CPU-bound work (argon2id, HMAC, image processing, reconciliation) have **three genuinely different runtime profiles**. Draft v1 put all three in one

process, then spent a page of \$5 explaining why password hashing was the system's first bottleneck. That bottleneck was self-inflicted by the architecture.

THE CORRECTION IN ONE LINE

Draft v1's own failure analysis was the argument against Draft v1. When the bottleneck section of a design says "the first thing to break is that these two workloads share a process," the design has told you where the seam belongs.

One thing I will not withdraw, because it is my job to say it: separating tiers raises the operational floor. That is a real cost and it appears in \$9 with a number. It is a cost worth paying here — but it should be paid knowingly, and the pilot plan needs to absorb it.

Design principles

Six. Each derives from a property of the business, and each one decides something concrete later.

#	PRINCIPLE	DERIVES FROM	WHAT IT DECIDES
1	The contract is the product	KAFRIADA intends to be the register other bodies trust and query.	OpenAPI 3.1 generated from the domain tier, versioned <code>/v1/</code> , published, with a generated typed client. No consumer — including our own UI — reaches past it.
2	Records are transactional; announcements are eventual	A transfer record is legal evidence. A news post about it is a courtesy.	One transaction writes the record and an outbox row. Workers turn outbox rows into posts, SMS and projections. Nothing that constitutes a record is ever eventually consistent.
3	Isolation is enforced by the database, not by discipline	The ledger and the audit log are the two artefacts whose integrity the whole product rests on.	Separate Postgres schemas, distinct least-privilege roles per code path, <code>INSERT</code> -only grants on audit, no <code>DELETE</code> anywhere near money. A compromised application cannot do what its role cannot do.
4	Every mutation is idempotent by contract	Users are on 2G. Requests time out and get retried. Money must not double.	<code>Idempotency-Key</code> is a first-class header on every mutating endpoint, not a special case bolted onto the Paystack webhook. Stored, replayed, expired.
5	Workload profiles do not share a runtime	Rendering a cached profile, minting a KUID and hashing a password are three different machines' worth of work.	Presentation, domain and worker are separate processes that scale on separate signals. CPU-bound work never competes with a latency-sensitive render path.
6	Boundaries drawn now, distribution deferred	The domain is one consistency domain today and may not be in three years.	Hard module boundaries with explicit contracts inside one deployable domain tier. Extraction later is a mechanical operation, not an excavation. See §5.

03 The shape

Four tiers, one consistency domain, one publicly reachable inbound path besides the UI.

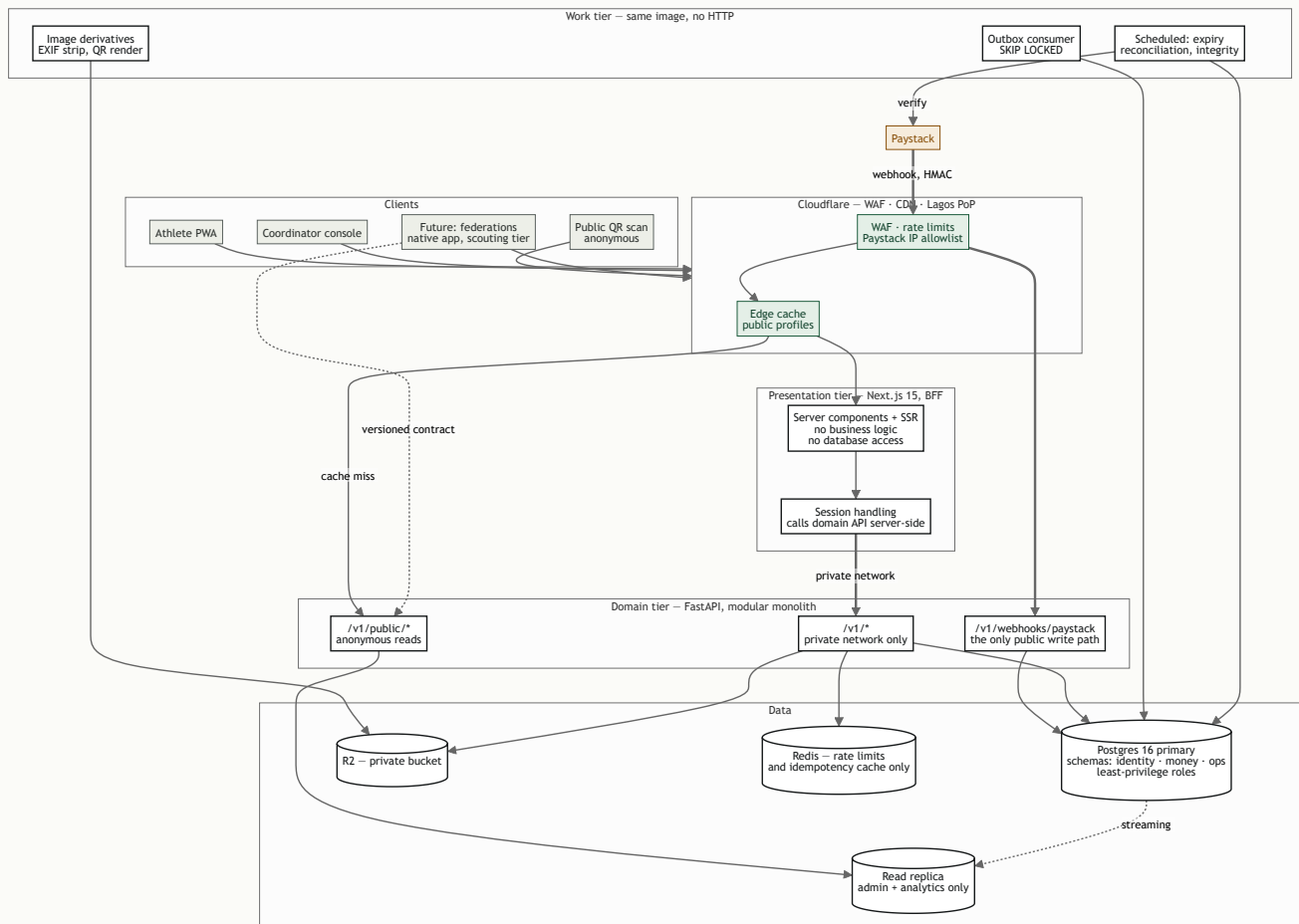


FIGURE 1 — TARGET TOPOLOGY. BOLD PATHS CARRY MONEY. THE DOMAIN API IS NOT INTERNET-REACHABLE EXCEPT FOR THE WEBHOOK AND THE ANONYMOUS READ SURFACE.

THE SECURITY CONSEQUENCE OF THE BFF PATTERN, WHICH IS THE BEST THING ABOUT THIS SHAPE

The browser talks only to the Next.js origin. Next.js calls the domain API **server-side over a private network**, forwarding the authenticated principal. Which means: **no CORS, no cross-origin cookie configuration, no session token ever in a browser-reachable API surface, and the entire authenticated API is not addressable from the internet at all**. The only publicly reachable domain-tier routes are the anonymous read surface and the Paystack webhook — and the webhook is IP-allowlisted and HMAC-authenticated.

That is a materially stronger posture than a single-tier application, where every route is one authorisation bug away from being public. Here, an authorisation bug on a private route is still unreachable by an attacker who has not already breached the presentation tier.

TIER	TECHNOLOGY	SCALES ON	HOLDS
Edge	Cloudflare — WAF, CDN, Lagos PoP	Automatic	Nothing. Caches the anonymous read surface for 60s.
Presentation	Next.js 15, App Router, TypeScript. Server components, server actions, PWA.	Concurrent renders	Session cookie handling, view composition, form handling. No business rule, no SQL, no direct database credential.
Domain	FastAPI, Python 3.12, SQLAlchemy 2.0 + Alembic, Pydantic v2. Uvicorn workers behind a process manager.	Request concurrency; CPU pool sized separately for hashing	Every business rule, every state machine, every authorisation decision, the only database credentials.
Work	Same image, no HTTP surface. Outbox consumer plus scheduler.	Queue depth	Everything asynchronous and everything CPU-heavy: reconciliation, expiry, image derivatives, notifications, projections, nightly integrity checks.
Data	Postgres 16 primary + streaming replica, PITR. R2 private. Redis for rate limits and idempotency cache only.	Vertical, then replica reads	Truth. Three schemas, distinct roles.

04 Why this split, derived from the system

Five arguments. None of them mentions the team.

1 — THE API IS A PRODUCT SURFACE WITH ITS OWN CONSUMERS AND ITS OWN LIFECYCLE

The GDOC's roadmap contains federation integration, a scouting tier and a native application. All three are API consumers. When the API is generated as a side effect of a UI framework's route handlers, its contract is whatever the UI happened to need last sprint, and versioning it later means either breaking consumers or freezing the UI. **Generating OpenAPI from Pydantic models makes the contract an artefact you review in a pull request**, diff between versions, and hand to a third party. That is the difference between having an API and having endpoints.

2 — THE MONEY CORE NEEDS A PRIVILEGE BOUNDARY THE APPLICATION CANNOT TALK ITS WAY PAST

Draft v1 proposed protecting the ledger with lint rules and code review. Those protect against mistakes, not against a compromised process. In the target architecture the ledger lives in a `money` schema, and the only database role with `INSERT` on `ledger_entries` is the one the payment code path connects with. The presentation tier holds no database credential at all. **An attacker who owns the Next.js process cannot write a ledger row, because the process it would have to use does not have the grant.** That is a property of the deployment, not a promise in a document.

3 — THREE WORKLOADS, THREE SCALING SIGNALS

WORKLOAD	PROFILE	SCALES ON	WHAT CO-LOCATING IT COSTS
Public profile render	I/O-bound, cache-dominated, latency-critical, 90%+ of request volume	Cache hit ratio, then render concurrency	Sharing a process with hashing means a registration drive degrades every QR scan at the same venue.
Transactional API	Short transactions, lock-sensitive, correctness-critical	Connection pool and transaction duration	Sharing with rendering means a slow render holds a connection the ledger needs.
argon2id, HMAC, image work	CPU-bound, 50–300ms per unit, bursty	Dedicated CPU pool	This is Draft v1's bottleneck #1. In the target architecture it is a sized thread pool in the domain tier and a separate worker for images — it cannot starve anything else.

4 — THE HARD PARTS OF THIS SYSTEM ARE PYTHON'S STRONG SUIT

Not a preference — a match between the problem and the ecosystem. **Pydantic v2** gives a validated, typed boundary on money amounts, webhook payloads and state transitions, with the schema and the validation being the same artefact. **Alembic** produces reviewable, reversible, auditable migrations — which matters when a migration touches a ledger. **Integer and Decimal discipline** is native rather than a convention fighting a language whose only number is a float. And the Phase-2 work the GDOC implies — document verification, OCR of identity documents, scouting analytics, duplicate detection across a national register — is work Python does and Node does not.

5 — THE PRESENTATION TIER GETS TO BE GENUINELY GOOD AT PRESENTATION

Next.js earns its place on merit once it is not also pretending to be an application server: server components render the public profile as cacheable HTML with no client JavaScript required (which the Opera Mini constraint demands), server actions handle forms without a client bundle, and the PWA shell is a first-class capability rather than an afterthought. **It is the right tool for the job it is now actually doing.**

WHERE I WILL STILL ARGUE WITH YOU

Every argument above is real, and the split is right. It is not free: §9 states the cost honestly, and the largest line is not money — it is that **the contract between the tiers becomes an artefact that has to be maintained**, and a stale generated client produces a class of bug that a single-tier application simply cannot have. The mitigation in §7 is not optional decoration; it is what keeps this decision from turning into the thing that slows you down.

Why the domain tier is not microservices

Argued from the domain rather than assumed, since the GDOC asks for them and you have told me to design the best system.

Microservices buy independent deployability, independent scaling, and independent failure of components with genuinely divergent load profiles and ownership. The question is whether KAFRIADA's domain has those seams. It does not, and the reason is specific:

The domain's core operations span what would be the service boundaries, transactionally. Completing a transfer must, as one indivisible act, close one club membership, open another, write a career event, and record the transfer as complete. Confirming a payment must, as one indivisible act, write two ledger lines and advance a verification state machine. Split identity, clubs, transfers and money into separate services and every one of those becomes a distributed saga with compensating transactions.

A saga is a weaker guarantee than a transaction. It admits states that a transaction cannot: an athlete transiently at two clubs, a ledger credited against a verification that failed to advance. Those states are recoverable, and in a system where they are unavoidable a saga is the right answer.

THE REASON IT IS THE WRONG ANSWER HERE

KAFRIADA's entire product is records that cannot be quietly wrong. Trading a guarantee of correctness for a property — independent deployability — that the system does not currently need is a bad trade at any team size. It would still be a bad trade with thirty developers. Distribute a system when the coupling costs more than the coordination; not before.

So: a modular monolith in the domain tier, with the seams cut and maintained as though extraction were imminent. Modules communicate through explicit, typed application-service interfaces; no module reads another's tables; each module owns its schema objects; cross-module writes pass an explicit unit of work. Every module is one step from being a service — and that step is taken when a specific module earns it, with the reason written down.

THE ONE SEAM I WOULD CUT FIRST, WHEN THE TIME COMES

Ledger and payments. Not for scale — for governance. The day KAFRIADA is large enough to be examined by a regulator, an auditor or an acquirer, a separately deployed money service with its own database, its own access log and its own release process is worth substantially more than a well-organised module. That day is not in this pilot, and the schema separation in principle 3 is what makes it a two-week job rather than a two-quarter one when it arrives.

Bounded contexts

Eleven, in three schemas. Draft v1 had six modules; the domain has more joints than that, and the ones it was hiding are the expensive ones.

Two changes from Draft v1 worth calling out before the table. **Verification is its own context, not part of identity** – identity answers "who is this person and what is their permanent number", verification answers "is this claim substantiated, by whom, and is that decision still standing". Those have different lifecycles, different actors, different SLAs, and verification is the monetised surface. Collapsing them is why Draft v1's state machine had no way to revoke an approval: the context that owns the decision was not distinct from the context that owns the person.

And **access is its own context**, not a utility folder. Authentication, authorisation and scope are the system's most security-critical logic; they deserve a boundary and an owner, not a shared drawer.

CONTEXT	SCHEMA	ANSWERS EXACTLY ONE QUESTION	OWNS
identity	identity	Who is this person, and what is their permanent number?	Athletes, the KUID counter, career history. The sole authority on KUID minting.
verification	identity	Is this claim substantiated, by whom, and does that decision still stand?	Verification cases, submissions, reviewer decisions, revocations, the resubmission counter.
clubs	identity	Who plays for whom, right now?	Clubs, rosters, org-account status. The at-most-one-open-membership invariant, as a partial unique index.
transfers	identity	How did an athlete move, and who agreed?	The transfer state machine and its evidence trail.
ledger	money	What money has this system ever seen?	Wallets, insert-only entries in integer kobo. No code path outside this context holds an INSERT grant.
payments	money	What did a provider tell us, and have we acted on it exactly once?	Payment intents, provider events, idempotency keys, reconciliation state. The only context that talks to Paystack.
access	ops	Who is this caller and what may they do, to what?	Users, credentials, sessions, roles, permissions and scope . Every authorisation decision in the system.
geography	ops	Where is this, and is it live?	States, LGAs, rollout flags. Small, stable, and depended on by almost everything.
feed	ops	What should this person be told about?	Posts and their targeting. The most isolated context in the system.
media	ops	Where does this file live and who may see it?	Asset records, presign issuance, derivative state. Enforces the paywall on reads.
audit	ops	What happened, who did it, and when?	Insert-only audit log. Write-only to every role; readable by two.

THE RULES BETWEEN THEM

- A context exposes an **application service** with typed inputs and outputs. Nothing else in it is importable. Enforced by an import-linter contract in CI, not by convention.

- No context reads another's tables. Cross-context reads go through the service; cross-context writes pass an explicit unit of work so they join the caller's transaction.
- **Dependencies point one way.** `access` , `geography` and `audit` are depended on and depend on nothing. `ledger` depends on nothing but `audit` — it is deliberately the most isolated context that does real work.
- `payments` may call into `verification` to advance a case on confirmed money. `verification` may never call into `payments` to ask whether something was paid — it is told, once, and records the fact. **One direction only, so there is exactly one definition of "paid".**

Five mechanisms that carry the system

The parts that have to be built right in the first fortnight, because everything else assumes them.

1 — Contract-first, machine-checked

The domain tier's Pydantic models generate OpenAPI 3.1. A build step generates a typed TypeScript client from that schema into the presentation tier. Three CI gates keep it honest:

- The committed OpenAPI document must match the one generated from source, or the build fails. The contract cannot drift silently.
- A breaking change to a `/v1/` schema fails the build unless the pull request carries an explicit version-bump marker. **Additive changes are free; removals and type changes require intent.**
- The presentation tier compiles against the generated client. A domain change that breaks a caller breaks the build, in the same pull request — which is the property that makes a two-tier system as safe to change as a one-tier one.

This is the mitigation for §4's honest warning. Without these three gates the split is a liability; with them, the contract is the strongest guarantee in the system.

2 — The transactional outbox

The mechanism that lets records stay atomic while side effects stay decoupled and retryable — and the reason this design needs no message broker.

```
-- Inside the same transaction that writes the record:
BEGIN
    ... write the domain record (transfer completed, payment confirmed) ...
    INSERT INTO ops.outbox (event_type, payload, available_at)
    VALUES ('transfer.completed', $1, now());
COMMIT

-- The worker, in a loop, with no broker involved:
BEGIN
    SELECT * FROM ops.outbox
    WHERE processed_at IS NULL AND available_at ≤ now()
    ORDER BY id
    FOR UPDATE SKIP LOCKED          -- many workers, no coordination, no duplicates
    LIMIT 20;
    ... handle: publish feed post, enqueue SMS, build projection ...
    UPDATE ops.outbox SET processed_at = now() WHERE id = ANY($ids);
COMMIT
```

WHY THIS AND NOT REDIS, RABBITMQ OR SQS

A broker cannot participate in your database transaction. That produces the oldest bug in event-driven systems: the transaction commits and the publish fails, so the record exists and the world is never told — or the publish succeeds and the transaction rolls back, so the world is told about something that did not happen. **The outbox makes "the record was written" and "the event exists" the same commit.** Delivery is then a retry loop over a table, which is simpler, inspectable with SQL, and survives every worker being dead for six hours. A broker is a correct addition when you need fan-out to systems that do not share your database. You do not.

3 — Least privilege, enforced by Postgres

The application connects with several distinct roles, chosen by code path. This is what makes the isolation claims in §2 and §4 true rather than aspirational.

ROLE	USED BY	GRANTS
kaf_app	Ordinary domain routes	SELECT/INSERT/UPDATE on <code>identity</code> and <code>ops</code> . SELECT on <code>money</code> . No write on <code>money</code> . No <code>DELETE</code> on <code>audit</code> .
kaf_money	Payment confirmation and reconciliation only	INSERT on <code>money.ledger_entries</code> , UPDATE on <code>money.payments</code> , plus what it needs in <code>identity</code> to advance a verification case. Nothing else in the codebase can acquire this connection.
kaf_reader	Public read surface, Metabase, analytics	SELECT only, and only on the replica. A reporting query cannot touch the write path, by construction.
kaf_migrate	Release step only	DDL. Never held by a running process.

No role anywhere holds `DELETE` or `UPDATE` on `ops.audit_log` or on `money.ledger_entries`. Not the application, not the migration role, not `super_admin` acting through the application. **"Insert-only" stops being a claim about our code and becomes a fact about the database.** A migration test asserts it on every deploy.

4 — Idempotency as an API primitive

Not a special case for Paystack — a property of the contract. Every mutating endpoint accepts an `Idempotency-Key`; the presentation tier generates one per user intent and reuses it across retries. The key, the request fingerprint and the response are stored; a replay returns the original response without re-executing. A key reused with a *different* body is a 422, because that is a client bug and silently accepting it is how money doubles.

Users on 2G retry. Load balancers retry. Users double-tap Submit. In a system that mints permanent identifiers and moves money, **retry safety is a contract obligation, not a defensive trick in one handler.** The Paystack webhook then needs no special mechanism — it is one more idempotent endpoint, keyed on the provider's event reference.

5 — Traceability end to end

OpenTelemetry from the edge through Next.js, into FastAPI, down to the SQL statement, and out to the worker that processes the resulting outbox row. One trace ID follows a QR scan or a payment across every hop and every process boundary.

This is where the tier split repays its complexity: in a single-tier application "the request was slow" is a guess, and here it is a waterfall showing render, network, authorisation, three queries and a lock wait. The same ID is surfaced on user-facing error screens and stamped into every audit row written during the request, so a coordinator reading a reference down the phone lands you in the exact trace.

08 Deployment and change

One repository, three images, one region, one migration discipline.

- **Monorepo, three build targets:** `web/` (Next.js), `api/` (FastAPI), `worker/` (same image as api, different entrypoint). One repository because the OpenAPI contract is generated in one place and consumed in another, and a contract split across two repositories drifts within a month.
- **Frankfurt, one availability zone for app and primary database.** Nigeria to Frankfurt is ~90–130ms; the inter-tier hop inside the zone is under 2ms and is not a meaningful part of any latency budget.
- **Migrations are a gated release step** under `kaf_migrate`, and must be backwards-compatible with the currently-running version. Expand, deploy, contract — so a rollback never requires a database rollback. This is stricter than Draft v1 and it is the discipline that makes rolling deploys safe.
- **The domain tier deploys independently of the presentation tier**, and that is a benefit, not an inconvenience: a UI change no longer redeploys the money path, and the money path can carry a heavier release gate — two reviewers, a mandatory integrity check, a deploy window — without slowing UI iteration.
- **Environments:** local via Compose with MinIO standing in for R2 and Paystack test keys; staging identical in shape and one size smaller, holding no real personal data; production as drawn.

What this costs, honestly

Stated plainly so the decision is made with the price visible.

INFRASTRUCTURE

Roughly **\$185–255/month** against Draft v1's \$97–142. The additions are the read replica (~\$25), a second application service (~\$30), Redis (~\$15), and higher aggregate compute. Not a material sum against what the architecture buys.

CONTRACT MAINTENANCE

The real cost. A generated client can go stale, and a stale client is a bug class a single-tier app cannot have. Fully mitigated by \$7.1's three CI gates — **but those gates are load-bearing, not optional**. If they are skipped under schedule pressure, this decision turns into the thing that slows you down.

OPERATIONAL SURFACE

Three deployables instead of one, two languages, two dependency trees, and a deploy order that matters (API before web, migrations before API). Runbooks and CI must exist before wave 1, not after.

SCHEDULE

Roughly **one additional week** of setup against the eight-week pilot: contract tooling, the role/grant migration, tracing, and a deploy pipeline for three targets. **RISK REGISTER** This lands on the plan, not on the architecture — but it does need to land somewhere, and Sprint 0 is where it belongs.

THE TRADE IN ONE SENTENCE

You are buying a defensible money boundary, an API that can be handed to a federation, three workloads that cannot starve each other, and a system that can be extracted into services without excavation — and paying for it with about \$110 a month, one week of Sprint 0, and a permanent obligation to keep three CI gates green.

What carries over, and what I need decided

The tier decision changes the shape. It changes almost none of the domain analysis.

CARRIES OVER FROM DRAFT V1 UNCHANGED

All fifteen findings in the challenge section, and every one of the flows. Specifically: argon2id outside the mint transaction and the atomic `INSERT ... ON CONFLICT ... RETURNING` counter, with the same concurrency proof; the phone number as identity anchor; `revoked` and `escalated` on the verification machine; a confirmed payment always beating expiry; gross and provider fee as separate ledger lines; private R2 with presigned reads and EXIF stripping; the versioned QR signature; assisted-payment caps and the athlete-facing SMS receipt; the reconciliation disagreement matrix; the eight-point restore drill; the permission matrix and its generated test suite; the phased LGA rollout with numeric gates; and the NDPR filing obligation above 2,000 subjects.

That is the point worth making about this revision: **the tier decision is a delivery-shape decision. The domain analysis was independent of it and survives intact.**

CHANGED BY THIS ARCHITECTURE

- Draft v1's "bottleneck #1 is argon2id" becomes a sized CPU pool in the domain tier rather than a structural limit.
- Feed posts and notifications move from in-transaction writes to outbox events. Career events stay in the transaction — they are records, not announcements.
- Redis returns, narrowly: rate limiting and the idempotency cache. **Not as a job broker** — that is the outbox, for the correctness reason in §7.2.
- Module isolation moves from ESLint conventions to Postgres grants plus an import-linter contract.
- A read replica appears, justified by workload isolation rather than volume.

FOUR DECISIONS I NEED FROM YOU BEFORE I WRITE THE FULL DESIGN

- 1 **Do you want the ledger extracted as a separate service in the pilot, or is schema-and-role separation sufficient for now?** My recommendation is schema separation now with the seam maintained — the governance value of a separate service is real but arrives later, and the schema boundary is what makes that extraction cheap. Say the word if you want it split from day one; the cost is a saga on payment confirmation and I would want to design that carefully.
- 2 **Is the public read surface part of the versioned `/v1/` contract from launch, or internal until a federation actually asks?** Publishing it early is a commitment to stability; keeping it internal costs a version bump later. I lean towards publishing it, because it is the surface that makes KAFRIADA infrastructure rather than an app — but that is a business call about how soon you want third parties depending on you.
- 3 **Confirm SQLAlchemy 2.0 + Alembic over the alternatives.** It is the boring, auditable choice and it handles the explicit locking and raw SQL the money paths need. I would not use an ORM that hides

the SQL on a ledger.

4

The remaining commercial questions from Draft v1 are still open and still block nothing technical: whether ₦2,500 is the price or the take, NIN in or out, who approves verifications and under what conflict rule, the refund policy, and who owns the NDPR filing.

ON THE CORRECTION YOU MADE

You were right about the derivation order, and the design is better for it. Deriving from the system rather than from capacity produced a stronger money boundary, an API that can become infrastructure, and the removal of a bottleneck that the previous draft could only document rather than eliminate. **The one thing I will keep saying is that delivery capacity does not stop being real because it stopped being an input** — it moves to the risk register, where \$9's extra week now sits, and it is managed there rather than allowed to quietly reshape the architecture.